

迷你应用

把你服务器上的一张网页挂进企业微信自建应用的**主页**与**底部菜单**，微信插件把它原样端进个人微信。一按就开，用起来像小程序，却不需要 ICP 备案的域名，也不需要营业执照。

// 不需要小程序账号 // 钥匙只开一组接口 // 底部菜单已在个人微信验证

```
个人微信 - 底部菜单「面板」 └─┐
个人微信 - 卡片「打开」   ─┬─> https://bot.example.com/mini?k=<MINI_APP_TOKEN>
企业微信 - 应用主页       ─┬─┐
                             |
                             └─ GET /mini           静态页面
                             └─ GET /api/mini/status  列表 JSON
                             └─ GET /api/mini/stream  SSE 实时流

只有 /api/mini/* 认这把钥匙，其余路径一律 401
```

企业微信负责「入口」

应用主页、底部菜单、卡片消息接受任意 HTTPS 链接。三处指向同一张页面。

你的服务器负责「内容」

一张静态 HTML 加两个同源 JSON 接口。列表、详情、实时流全在里面。

钥匙负责「不越界」

独立的一把钥匙只认 /api/mini/ 前缀。链接泄露也打不开别的门。

01 / PRINCIPLE

小程序有两堵墙，自建应用一堵都没有

微信小程序能请求的域名必须完成 ICP 备案；个人主体不能使用 web-view，企业主体又要营业执照。企业微信自建应用的主页、菜单和卡片对链接没有这些要求，域名照旧走主教程第三步配好的隧道。

你得到的是什么。

在个人微信里点开机器人的会话，输入框左侧多出一个菜单；按一下，微信自带的浏览器打开你服务器上的页面。进程观察窗、状态面板、日志尾部、遥控器，都可以是这张页面。本文把它叫做迷你应用。

三样东西，一把钥匙

全部跑在现有机器人服务器上。同源，所以不需要 CORS，EventSource 也能直接用。

PAGE	code/mini_app_page.html · 路径 /mini。钥匙引导、列表、详情、实时流、暂停恢复，全部内联，没有外部依赖
API	/api/mini/status 返回列表 JSON，/api/mini/stream 是 SSE 实时流。参考实现 code/mini_app_server.py ，标准库
KEY	环境变量 MINI_APP_TOKEN，mode 600 文件。只在路径以 /api/mini/ 开头时才认
WECOM	code/mini_app_setup.sh · 三次 API 调用把链接写进应用主页、底部菜单与一张卡片
TEST	tests/test_mini_app.py · 正确钥匙敲别的路径必须失败

钥匙只开一小块

链接会被存进腾讯侧的菜单配置和聊天记录里，所以链接里的钥匙必须只能打开这一组接口。规则只有一条：从 ?auth= 或 Authorization: Bearer 取钥匙，恒定时间比较，且只在路径以 /api/mini/ 开头时才认。

```
import hmac

API_PREFIX = "/api/mini/"

def is_scoped_request(path: str, presented: str, expected: str) -> bool:
    """Only paths under API_PREFIX accept the mini-app token."""
    if not expected or not presented:
        return False
    if not path.startswith(API_PREFIX):
        return False
    return hmac.compare_digest(presented.encode("utf-8"), expected.encode("utf-8"))
```

```

const crypto = require("crypto");

const MINI_APP_TOKEN = process.env.MINI_APP_TOKEN || "";

function safeEqual(a, b) {
  const x = crypto.createHash("sha256").update(String(a)).digest();
  const y = crypto.createHash("sha256").update(String(b)).digest();
  return crypto.timingSafeEqual(x, y);
}

function miniAppAuth(req, res, next) {
  const presented = (typeof req.query.auth === "string" && req.query.auth) ||
    (req.headers.authorization || "").replace(/^Bearer\s+/i, "");
  const scoped = req.path.startsWith("/api/mini/");
  if (MINI_APP_TOKEN && scoped && safeEqual(presented, MINI_APP_TOKEN)) return next();
  return res.status(401).json({ error: "unauthorized" });
}

app.get("/mini", (req, res) => res.sendFile(path.join(__dirname, "mini_app_page.html")));
app.get("/api/mini/status", miniAppAuth, (req, res) => res.json({ items: loadStatus() }));

```

如果接口里有会动手的操作。

只读的观察页用一把钥匙足够。一旦 /api/mini/ 下面出现重启进程、发送按键这类会改变服务器状态的接口，拿到链接的人也能按这些按钮。至少做到危险操作二次确认、每次调用记日志；更稳的做法是把动手类接口放到另一把钥匙后面，不写进任何链接。生成钥匙用 `openssl rand -hex 24`；换钥匙改一行、重启服务、重跑配置脚本，旧链接立刻失效。

页面第一次打开时把钥匙收好

链接带着 ?k= 来一次，页面存进 localStorage，之后每次请求拼成 ?auth=，地址栏擦干净。微信内置浏览器对同一域名的本地存储是持久的。

01 读钥匙

从 location.search 读 k，同时兼容 #k=。没有就看 localStorage；两处都空，显示一个输入框让人粘贴。

02 擦地址栏

history.replaceState 把地址换成干净的 /mini。截图、分享、浮窗里都不再带钥匙。

03 带钥匙请求

fetch 与 EventSource 都用 ?auth=。EventSource 不能自定义请求头，这也是服务端同时接受查询参数与 Bearer 头的原因。

04 401 就回到输入框

钥匙换了、本地存储被清了，页面清空旧钥匙回到引导页。从菜单再点一次，链接自带的 ?k= 会重新引导。

用查询参数，不要用井号。

微信打开卡片和菜单链接时可能丢掉 # 后面的片段。参考页面两种都认，但写进企业微信的链接一律用 ?k=。

05 / WECOM

三次调用，把链接写进腾讯侧

CORP_ID 与 APP_SECRET 来自主教程第一、二步，AGENT_ID 是自建应用的数字 ID，USER_ID 是你自己的成员账号，PAGE_URL 形如 `https://bot.example.com/mini?k=<MINI_APP_TOKEN>`。三次都以 "errcode": 0 为准。

```
curl -s "https://qyapi.weixin.qq.com/cgi-bin/gettoken?corpid=${CORP_ID}&corpsecret=${APP_SECRET}"
```

access_token 两小时有效，长进程里要缓存复用


```
curl -s -X POST "https://qyapi.weixin.qq.com/cgi-bin/agent/set?access_token=${ACCESS_TOKEN}" \  
-H "Content-Type: application/json" \  
-d "{\"agentid\": ${AGENT_ID}, \"home_url\": \"${PAGE_URL}\"}"
```

```
curl -s -X POST "https://qyapi.weixin.qq.com/cgi-bin/menu/create?
access_token=${ACCESS_TOKEN}&agentid=${AGENT_ID}" \
-H "Content-Type: application/json" \
-d "{\"button\": [{\"type\": \"view\", \"name\": \"面板\", \"url\": \"${PAGE_URL}\"}]}"

# 顶级按钮名称限 16 字节，大约五个汉字
```

```
curl -s -X POST "https://qyapi.weixin.qq.com/cgi-bin/message/send?access_token=${ACCESS_TOKEN}" \
-H "Content-Type: application/json" \
-d '{"touser": "${USER_ID}", "msgtype": "textcard", "agentid": ${AGENT_ID},
  "textcard": {"title": "面板", "description": "点开就是。长按可以收进浮窗。",
  "url": "${PAGE_URL}", "btntxt": "打开"}}'
```

MENU	个人微信里点开机器人会话，输入框左侧的菜单图标展开就是你的按钮。已验证可见，也是日常最顺手的入口
CARD	出现在会话流里，标题、描述、「打开」按钮。长按打开的页面可以收进微信浮窗
HOME	在企业微信客户端的工作台与应用页里生效。微信插件侧是否展示未验证，不要依赖它
NOTICE	个别域名首次打开会看到「非微信官方网页」的提示，点继续即可，不影响功能

06 / VERIFY

接口说 OK 还不够，手机里真的出现才算通

先在桌面浏览器直接打开 PAGE_URL 确认页面本身能出来，再到微信里逐项勾。

- | | |
|--|---|
| ☐ 桌面浏览器能打开页面
列表出来了，地址栏里的 ?k= 已经被擦掉。 | ☐ 底部菜单在个人微信里可见
点开机器人会话，输入框左侧有菜单，按下去页面打开。 |
| ☐ 卡片能点开
「打开」按钮进入同一张页面，长按可收进浮窗。 | ☐ 实时流在动
点进一项，快照随内容变化刷新，切走再切回会自动重连。 |
| ☐ 钥匙越界被拒
用同一把钥匙请求 /api/mini/ 之外的路径，返回 401。 | ☐ 换钥匙后旧链接失效
改环境变量、重启、重跑脚本，旧卡片打开是引导页，新菜单直接进。 |

七个最常见的断点

先确认页面本身，再确认钥匙，最后才是微信侧。

BLANK	菜单点开是空白页或 404。	PAGE_PATH 与链接不一致，或静态文件路径不对。先用桌面浏览器直接打开 PAGE_URL。
SPINNING	页面出来了，列表一直转圈。	钥匙没进本地存储：链接用了 #k= 被微信丢了，改成 ?k=；或 MINI_APP_TOKEN 为空，服务器对所有请求 401。
401	电脑上正常，微信里 401。	微信内置浏览器的本地存储被清过。从菜单重新打开一次，链接自带钥匙。
STREAM	实时流几秒后断。	中间的反向代理开了缓冲。SSE 响应加 Cache-Control: no-cache 与 X-Accel-Buffering: no，服务端每十五秒发一条注释行保活。
40018	menu/create 报名称错误。	顶级按钮名称超过 16 字节。缩短到五个汉字以内。
60020	三次调用之一被拒。	服务器出口 IP 不在企业可信 IP 里，与主教程第九章处理方式相同。
STALE	换了钥匙后菜单打不开。	只改了环境变量没重跑配置脚本，菜单里还是旧链接。

公开一张页面，收紧四条边

这张页面会被微信浏览器缓存，也可能被截图。它能看到的東西，就是链接持有者能看到的東西。

钥匙只开一小块

只认 `/api/mini/`。把这条规则写成测试，正确钥匙敲别的路径必须失败。

只绑本机

服务绑定 `127.0.0.1`，通过隧道或反向代理对外，和主教程的回调服务一样。

钥匙不进日志

访问日志只记路径，不记查询串。参考实现的 `log_message` 就是这么写的。

页面里没有私人信息

名字、账号、路径、内部域名一律不写进页面。公开仓库只提交 `.env.example` 与占位链接。

动手类接口另开一把钥匙

重启、发键这类操作不要放在写进链接的那把钥匙后面。至少二次确认加日志。

换钥匙是一行的事

改 `MINI_APP_TOKEN`、重启、重跑 `mini_app_setup.sh`。菜单与主页立刻拿到新链接。

09 / SOURCES

官方接口索引

以企业微信当前文档与管理后台为准。

AGENT / SET	/document/path/90228
MENU / CREATE	/document/path/90231
MESSAGE / SEND	/document/path/90236
GETTOKEN	/document/path/91039

入口交给企业微信，
内容留在自己的服务器。
一把钥匙只开一扇小门。

© 2026 RORONOA & HARUKA · DOCS [CC BY-NC-SA 4.0](#) · CODE [POLYFORM NONCOMMERCIAL 1.0.0](#) · PLACEHOLDER IDS ONLY · github.com/RoronoaHaruka/wecom-companion-guide